

Autonomous Parking

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

1. Learn the actions the car takes after recognizing parking A
2. Learn the newly emerged key source code

2. Road Sign Detection

Road sign detection source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

3. Program Startup

3.1 Startup Command

- Parameter Configuration

Parameter path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml
```

```
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: False
DEBUG_MODE_2: True
START_AutoDrive: True
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

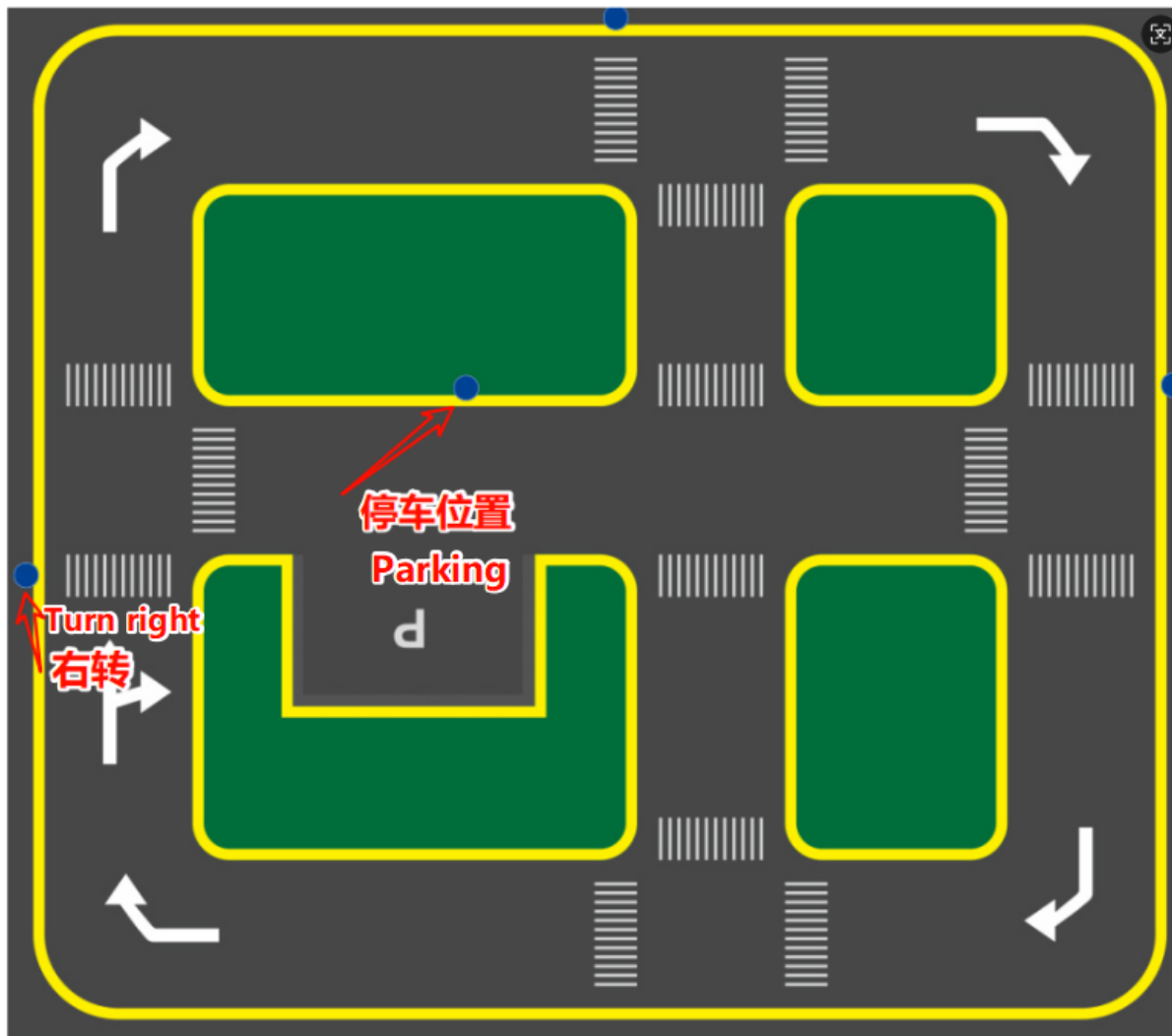
After modifying parameters, restart the launch file to use the modified parameters.

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

```
roslaunch auto_driver auto_drive.launch
```


3.2 Car movement after recognizing parking

Note: Because autonomous parking uses fixed actions, the right turn and parking A signs need to be placed at fixed positions



When the program recognizes parking A, the program will print (Parking A/Parking B perform the same action)

```
jetson@yahboom: ~  
jetson@yahboom: ~ 80x24  
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] Loaded engine size: 12 MiB  
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [W] Using an engine plan file across  
different models of devices is not recommended and is likely to affect performan  
ce or even cause errors.  
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [W] Using an engine plan file across  
different models of devices is not recommended and is likely to affect performa  
nce or even cause errors.  
[ascamera_node-1] [INFO] [1764671375.556312801] [ascamera_hp60c.camera_publisher  
]: 2025-12-02 18:29:35[INFO] [CameraHp60c.cpp] [1148] [streamCallback] set gain  
ret 0, gain 4  
[auto_drive-4] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-managed  
allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28 (MiB)  
[yolo_detect-3] [12/02/2025-18:29:35] [TRT] [I] [MemUsageChange] TensorRT-manage  
d allocation in IExecutionContext creation: CPU +0, GPU +19, now: CPU 0, GPU 28  
(MiB)  
[auto_drive-4] [INFO] [1764671378.387579116] [auto_drive]: current_sign redlight  
[auto_drive-4] [INFO] [1764671378.390984163] [auto_drive]: red_stop  
[auto_drive-4] [INFO] [1764671832.607130196] [auto_drive]: current_sign greenlig  
ht  
[auto_drive-4] [INFO] [1764671832.608561506] [auto_drive]: greenlight  
[auto_drive-4] [INFO] [1764673308.321102763] [auto_drive]: current_sign parkA  
[auto_drive-4] [INFO] [1764673308.323742671] [auto_drive]: parking A
```

The car will execute fixed actions to enter the parking space, first moving forward for 3 seconds, then parking laterally



The autonomous parking handling function is in the auto_drive.py file. Below is the function that is entered after recognizing parking. Users can modify the car's movement after recognizing the road sign themselves

```
def parking_A(self):
    rospy.loginfo('Parking A')
    time.sleep(5.0)
    self.flag = True
    self._set_current_speed(linear_x=-self.drive_speed, angular_z=0.8)
    time.sleep(3.6)
    self._set_current_speed(linear_x=-self.drive_speed, angular_z=-0.8)
    time.sleep(3.0)
    self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
    time.sleep(2.2)
    self._set_current_speed(linear_x=0.0, angular_z=0.0)
```

4. Source Code Analysis

4.1, auto_drive.py

YOLO signpost message reception:

```
results = self.sign_detect.get_infer_result(frame, True)
annotated_frame = frame.copy()
```

Multiple filtering mechanisms: mainly to prevent continuously recognizing road signs and triggering actions; the action will only be unlocked when different road signs are recognized.

```
# If a sign that requires cooldown is detected, start cooldown
if detected_sign_requires_cooldown:
    self.sign_cooldown = True
    self.last_sign_time = current_time
    rospy.loginfo(f"Sign detected, starting {self.cooldown_duration}s cooldown")
# Check if the repeated sign should be ignored
should_ignore = False
if obj.class_name == self.last_sign:
    should_ignore = True
```

Event queue processing

```
def _event_handling_worker(self):
    '''Event handling worker thread'''
    # sign handling mapping table
    sign_handlers = {
        "straight": self.go_straight,
        "turnright": lambda: self.turn('right'),
        "turnright_ground": self.do_nothing,
        "honking": self.car_horn,
        "sidewalk": self.slow_down,
        "sidewalk_ground": self.do_nothing,
        "speedlimit": self.slow_down,
        "stop": self.stop,
        "school": self.school,
        "parkA": self.parking_A,
        "parkB": self.parking_A,
        "greenlight": self.go_straight,
        "redlight": self.red_stop,
        "yellowlight": self.stop,
        "out_park": self.out_parking
```

```

    }

    # sign enable index mapping
    sign_enable_map = {
        "straight": 10, "turnright": 11, "honking": 1, "sidewalk": 6,
        "sidewalk_ground": 7, "speedlimit": 8, "stop": 9, "school": 5,
        "parkA": 2, "parkB": 3, "greenlight": 0, "redlight": 4,
        "yellowlight": 13
    }

```

Specific action implementation

```

def parking_A(self):
    rospy.loginfo('Parking A')
    time.sleep(5.0)
    self.flag = True
    self._set_current_speed(linear_x=-self.drive_speed, angular_z=0.8)
    time.sleep(3.6)
    self._set_current_speed(linear_x=-self.drive_speed, angular_z=-0.8)
    time.sleep(3.0)
    self._set_current_speed(linear_x=self.drive_speed, angular_z=0.0)
    time.sleep(2.2)
    self._set_current_speed(linear_x=0.0, angular_z=0.0)

```